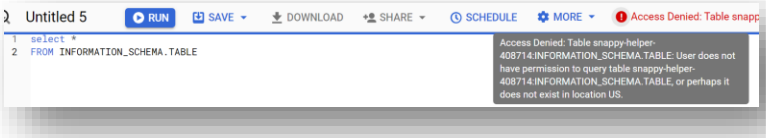
Q1. **Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:**

A. Data type of all columns in the "customers" table.

Ans:

Getting this error while running this query –

```
select * FROM INFORMATION_SCHEMA.TABLE
```



So moved forward with below query –

```
select
customer_id,
customer_unique_id,
customer_zip_code_prefix,
customer_city, customer_state
FROM Target_SQL.customers
```

Customer table consist of columns :

- customer_id – String
- customer_unique_id – String
- customer_zip_code_prefix - Integer
- customer_city – String
- customer_state – String

Output -

Row	customer_id	customer_unique_id	customer_zip_code_	customer_city	customer_state
1	0735e7e4298a2ebbb46649346...	fcb003b1bdc0df64b4d065d9b...	59650	acu	RN
2	903b3d86e3990db01619a4ebe...	46824822b15da44e983b021d...	59650	acu	RN
3	38c97666e962d4fea7fd6a83e...	b6108acc674ae5c99e29adc10...	59650	acu	RN
4	77c2f46cf580f4874c9a5751c2...	402cce5c0509000eed9e77fec...	63430	ico	CE
5	4d3ef4cfffb8ad4767c199c36a...	6ba00666ab7eada5ceec279b2...	63430	ico	CE
6	3000841b86e1fbe9493b52324...	796a0b1a21f597704057184a1...	63430	ico	CE
7	3c325415ccc7e622c66dec4bc...	05d1d2d9f0161c5f397ce7fc77...	63430	ico	CE
8	04f3a7b250e3be964f01bf22bc...	c34585a0276ecc5e4fb03de75...	63430	ico	CE
9	894202b8ef01f4719a4691e79...	01a4fe5fc00bbdb0b0a4af5a53...	63430	ico	CE
10	9d715b9fb75a9d081c14126c0...	8f399f3b7ace8e6245422c9e1f...	63430	ico	CE

Insights - customer_zip_code_prefix is only integer present in Customer Table. No null are present in Customer Table.

Recommendation – N/A

B. Get the time range between which the orders were placed.

Ans:

Query –

```
select
max(order_purchase_timestamp) as Max_time_range,
min(order_purchase_timestamp) as Min_time_range
from `Target_SQL.orders`
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON

Row	Max_time_range	Min_time_range
1	2018-10-17 17:30:18 UTC	2016-09-04 21:15:19 UTC

Insights – Target is beginning its operation on 2016-09-04 21:15:19 UTC till 2018-10-17 17:30:18 UTC and under this, purchase made by customers

Recommendation – N/A

C. Count the Cities & States of customers who ordered during the given period.

Ans:

Query –

```
select
count (distinct(customer_city)) as customer_city,
count (distinct(customer_state)) as customer_state
from snappy-helper-408714.Target_SQL.customers
order by customer_state
```

Output –

Query results

JOB INFORMATION RESULTS CHART

Row	customer_city	customer_state
1	4119	27

Insights - customer_city and customer_state having similar count of 99441 which customer have ordered.

Recommendation – N/A

2. In-depth Exploration:

2A. Is there a growing trend in the no. of orders placed over the past years?

Ans:

Query –

```
select
count(*) as Number_of_orders,
extract(year from order_purchase_timestamp) as purchase_year
from `Target_SQL.orders`
group by purchase_year
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON

Row	Number_of_orders	purchase_year
1	45101	2017
2	54011	2018
3	329	2016

Insights – Data contains number of order year wise, number of order increases in 2018 from 2016, 2017.

Recommendation – N/A

2B. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Ans:

Query –

```
select
count(*) as Number_of_orders,
extract(year from order_purchase_timestamp) as purchase_year,
extract(month from order_purchase_timestamp) as purchase_month
from `Target_SQL.orders`
group by purchase_month, purchase_year
order by purchase_year, purchase_month
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JS

Row	Number_of_orders	purchase_year	purchase_month
1	4	2016	9
2	324	2016	10
3	1	2016	12
4	800	2017	1
5	1780	2017	2
6	2682	2017	3
7	2404	2017	4
8	3700	2017	5
9	3245	2017	6
10	4026	2017	7

Insights – Data contains number of orders month wise along with year in ascending order.

Recommendation – N/A

2C. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

Ans-

Query –

```
select
count(*) as Number_of_orders,
case
WHEN Extract (hour FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN
      'dawn'
      WHEN Extract (hour FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN
      'morning'
      WHEN Extract (hour FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN
      'Afternoon'
      WHEN Extract (hour FROM order_purchase_timestamp) BETWEEN 19 AND 23 THEN
      'Night'
       END as timing
from `Target_SQL.orders`
group by timing
order by Number_of_orders
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW

Row	Number_of_orders	timing
1	5242	dawn
2	27733	morning
3	28331	Night
4	38135	Afternoon

Insights – In afternoon, the greatest number of order place, we have more active customer in afternoon and in Night.

Recommendations – we have target customer with help of more offers and clearance sale , we can target the dawn customer.

3. Evolution of E-commerce orders in the Brazil region:

3A. Get the month-on-month no. of orders placed in each state.

Ans:

Query –

```
select c.customer_state,
    extract (month from o.order_purchase_timestamp) as order_month,
    count(*) as monthwise_orders
  from snappy-helper-408714.Target_SQL.customers c join snappy-helper-
408714.Target_SQL.orders o
  using (customer_id)
  group by c.customer_state,order_month
  order by c.customer_state,order_month
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON

Row	customer_state	order_month	monthwise_orders
1	AC	1	8
2	AC	2	6
3	AC	3	4
4	AC	4	9
5	AC	5	10
6	AC	6	7
7	AC	7	9
8	AC	8	7
9	AC	9	5
10	AC	10	6

Insights – Customer state wise, data of month wise orders under which month order made b customer.

Recommendations – N/A

3B. How are the customers distributed across all the states?

Ans:

Query –

```
select geolocation_state,
count(DISTINCT (customer_id)) as customers,
  from snappy-helper-408714.Target_SQL.geolocation g join snappy-helper-
408714.Target_SQL.customers c
  on g.geolocation_zip_code_prefix = c.customer_zip_code_prefix
  group by geolocation_state
  order by geolocation_state
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART | PREVIEW

Row	geolocation_state	customers
1	AC	120
2	AL	412
3	AM	148
4	AP	68
5	BA	3371
6	CE	1332
7	DF	1974
8	ES	2027
9	GO	2011
10	MA	743

Insights – data contains, number of customers present in each state.

Recommendation – with help of this data, we can target those state where customer numbers are less. And with of ads, we can acquire more potential customer.

4. Impact on Economy: Analyse the money movement by e-commerce by looking at order prices, freight and others.

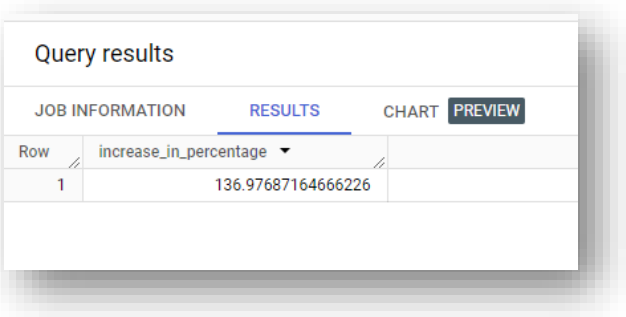
4A. Get the % increase in the cost of orders from year 2017 to 2018 *(include months between Jan to Aug only).*

Ans:

Query –

```
select
((data_2018 - data_2017)/data_2017)
*100 as increase_in_percentage
FROM
(
select
SUM
(
CASE
WHEN extract(year from o. order_purchase_timestamp) = 2017 AND extract(month from
o.order_purchase_timestamp) BETWEEN 01 AND 08 THEN payment_value
ELSE 0
END
) as data_2017,
SUM
(
CASE
WHEN extract (year from o.order_purchase_timestamp) = 2018 AND extract(month from
o.order_purchase_timestamp) BETWEEN 01 AND 08 THEN payment_value
ELSE 0
END
)as data_2018
from Target_SQL.orders o join Target_SQL.payments p ON o.order_id = p.order_id
where extract(year from o.order_purchase_timestamp) IN (2017, 2018) AND
extract(month from o.order_purchase_timestamp) BETWEEN 01 AND 08
)
```

Output –



Row	increase_in_percentage
1	136.97687164666226

Insights – in data, we have found that how much there is increase in percentage in cost of orders in 2017 and 2018 from Jan to Aug month.

Recommendations – we can target for the next coming year how can we more improve on the cost of orders with the help of

4B. Calculate the Total & Average value of order price for each state.

Ans:

Query –

```
select
round (sum(price),2) as total_price,
round (avg(price),2) as avg_price,
customer_state
from `Target_SQL.order_items` ot left join `Target_SQL.orders` o
using (order_id) left join `Target_SQL.customers` c using (customer_id)
group by customer_state
order by customer_state
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON

Row	total_price	avg_price	customer_state
1	15982.95	173.73	AC
2	80314.81	180.89	AL
3	22356.84	135.5	AM
4	13474.3	164.32	AP
5	511349.99	134.6	BA
6	227254.71	153.76	CE
7	302603.94	125.77	DF
8	275037.31	121.91	ES
9	294591.95	126.27	GO
10	119648.22	145.2	MA

Insights – data contains, total price and Average price of product as per state.

Recommendation – NA

4C. Calculate the Total & Average value of order freight for each state.

Ans:

Query –

```
select
round (sum(freight_value),2) as total_freight_value,
round (avg(freight_value),2) as avg_freight_value,
customer_state
from `Target_SQL.order_items` ot left join `Target_SQL.orders` o
using (order_id) left join `Target_SQL.customers` c using (customer_id)
group by customer_state
order by customer_state
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON

Row	total_freight_value	avg_freight_value	customer_state
1	3686.75	40.07	AC
2	15914.59	35.84	AL
3	5478.89	33.21	AM
4	2788.5	34.01	AP
5	100156.68	26.36	BA
6	48351.59	32.71	CE
7	50625.5	21.04	DF
8	49764.6	22.06	ES
9	53114.98	22.77	GO
10	31523.77	38.26	MA

Insights – data contains, total fright value and Average total fright value of product as per state.

Recommendations – NA

5. Analysis based on sales, freight and delivery time.

5A. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.
Also, calculate the difference (in days) between the estimated & actual delivery date of an order. Do this in a single query.

Ans:

Query –

```
select
extract(day from order_delivered_customer_date) - extract(day from
order_purchase_timestamp) as delivery_time,
extract(day from order_estimated_delivery_date) - extract(day from
order_delivered_customer_date) as diff_estimated_delivery
from `Target_SQL.orders`
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART | PRE

Row	delivery_time	diff_estimated_delivery
1	0	-1
2	0	2
3	-28	11
4	-29	12
5	21	-12
6	-27	13
7	-29	12
8	23	-17
9	-27	13
10	25	-24

Insight – data contains the delivery time and diff estimated delivery.

Recommendation – we can more focus on reducing the delivery time and diff estimated delivery.

5B. Find out the top 5 states with the highest & lowest average freight value.

Ans:

Query –

```
(
  select
  round (avg(freight_value),2) as avg_freight_value,
  customer_state
  from Target_SQL.order_items ot left join Target_SQL.orders o
   using (order_id) left join Target_SQL.customers c using (customer_id)
     group by customer_state
     order by avg_freight_value desc
     limit 5
)
union all
(select
  round (avg(freight_value),2) as avg_freight_value,
  customer_state
  from Target_SQL.order_items ot left join Target_SQL.orders o
   using (order_id) left join Target_SQL.customers c using (customer_id)
     group by customer_state
  order by avg_freight_value asc
limit 5 )
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW

Row	avg_freight_value	customer_state
1	42.98	RR
2	42.72	PB
3	41.07	RO
4	40.07	AC
5	39.15	PI
6	15.15	SP
7	20.53	PR
8	20.63	MG
9	20.96	RJ
10	21.04	DF

Insights – data contains five highest avg_freight_value and five lowest avg_freight_value.

Recommendations – NA

5C. Find out the top 5 states with the highest & lowest average delivery time.

Ans:

Query -

```
(
  select
avg(extract(day from order_delivered_customer_date) - extract(day from
order_purchase_timestamp)) as delivery_time,
customer_state
from `Target_SQL.orders` o left join Target_SQL.customers c using (customer_id)
group by customer_state,order_delivered_customer_date, order_purchase_timestamp
order by delivery_time desc
limit 5
)
UNION ALL
(
    select
avg(extract(day from order_delivered_customer_date) - extract(day from
order_purchase_timestamp)) as delivery_time,
customer_state
from `Target_SQL.orders` o left join Target_SQL.customers c using (customer_id)
where (extract(day from order_delivered_customer_date) - extract(day from
order_purchase_timestamp)) is not null
group by customer_state,order_delivered_customer_date, order_purchase_timestamp
order by delivery_time asc
limit 5
)
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW

Row	delivery_time	customer_state
1	-30.0	SP
2	-30.0	SP
3	-30.0	SP
4	-30.0	SP
5	-30.0	SP
6	30.0	RS
7	30.0	AL
8	29.0	PA
9	29.0	PA
10	29.0	BA

Insights – In data, from 1 to 5, it contains the lowest delivery time and 6 to 10, it contains the highest delivery time.

Recommendation – Company can more on highest delivery time states and can create nearby warehouses to ship the product in less delivery time.

5D. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Query –

```
WITH CTE AS
(
select *,
  dense_rank() OVER(order by t.average_delivery_time) as position
from
(
  select distinct c.customer_state,
    AVG(DATE_DIFF(order_delivered_customer_date,order_estimated_delivery_date,day))
OVER(PARTITION BY c.customer_state) as average_delivery_time
  from Target_SQL.orders o JOIN Target_SQL.customers c ON o.customer_id =
c.customer_id
  order by average_delivery_time
) t
order by position
)
select customer_state,average_delivery_time from CTE where position < 6 order by
average_delivery_time;
```

Output -

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON | EXECUTIO

Row	customer_state	average_delivery_time
1	AC	-19.7625
2	RO	-19.131687242798353
3	AP	-18.731343283582088
4	AM	-18.606896551724137
5	RR	-16.414634146341463

Insights – data contains, Statewise customer with fastest delivery for top 5 states.

Recommendations - NA

6. Analysis based on the payments:

6A. Find the month-on-month no. of orders placed using different payment types.

Ans:

Query –

```
select payment_type,
    extract (month from o.order_purchase_timestamp) as order_month,
    count(*) as monthwise_orders
  from Target_SQL.payments c join Target_SQL.orders o
  using (order_id)
  group by payment_type,order_month
  order by payment_type,order_month
```

Output –

Query results

JOB INFORMATION | RESULTS | CHART PREVIEW | JSON

Row	payment_type	order_month	monthwise_orders
1	UPI	1	1715
2	UPI	2	1723
3	UPI	3	1942
4	UPI	4	1783
5	UPI	5	2035
6	UPI	6	1807
7	UPI	7	2074
8	UPI	8	2077
9	UPI	9	903
10	UPI	10	1056
11	UPI	11	1509
12	UPI	12	1160
13	credit_card	1	6103

Insights – Data contains, types of payment methods use by customer, monthly wise and how much order we received in that month.

Recommendation - NA

6B. Find the no. of orders placed on the basis of the payment instalments that have been paid.

Ans:

Query –

```
select
    count(*) as no_of_orders,
    payment_installments
  from Target_SQL.payments P join Target_SQL.orders o
  using (order_id)
  Where payment_installments >= 1
  group by payment_installments
```

Output –

Query results

JOB INFORMATION | RESULTS | CHAR

Row	no_of_orders	payment_installment
1	52546	1
2	12413	2
3	10461	3
4	7098	4
5	5239	5
6	3920	6
7	1626	7
8	4268	8
9	644	9
10	5328	10
11	23	11

Insights – data contains, no of orders with no of payment installments.

Recommendation - NA